Special topics of AI

Done by:zaid abunassar 0223474

Problem Statement:

Imbalanced datasets often cause machine learning models to struggle, especially when it comes to accurately identifying minority classes.

This issue arises in many real-world applications such as medical diagnosis, fraud detection, and various classification tasks.

This project aims to tackle this problem by generating synthetic data samples for the minority class.

We use different types of Generative Adversarial Networks (GANs) to create these synthetic samples. To make the dataset more balanced through synthetic data augmentation.

Dataset Description & Imbalance Analysis

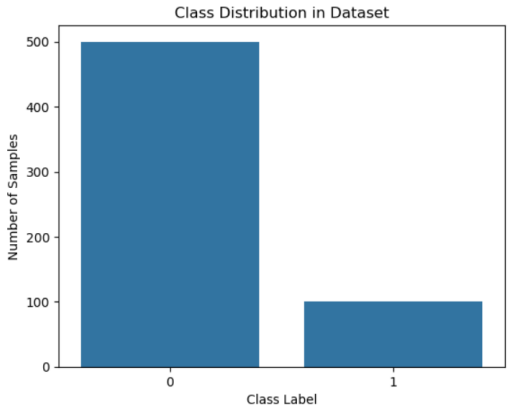
Dataset Description

The dataset used in this project is the Diabetes Dataset, which contains medical diagnostic measurements for patients. It consists of 600 samples with 8 numerical features related to health indicators such as glucose level, blood pressure, BMI, and age.

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	3	111	90	12	78	28.4	0.495	29	0
1	2	94	76	18	66	31.6	0.649	23	0
2	0	177	60	29	478	34.6	1.072	21	1
3	2	107	74	30	100	33.6	0.404	23	0
4	0	78	88	29	40	36.9	0.434	21	0

The target variable is binary, where class 0 represents non-diabetic cases and class 1 represents diabetic cases.

The dataset is highly imbalanced with fewer positive cases(100) compared to



negative(500)

Details of GAN Architectures & Training:

In this project, the default GAN (Vanilla GAN) was used, and I chose WGAN-GP and LSGAN. I selected them because they are considered the best for tabular data types. WGAN-GP is an upgraded version of WGAN. I tried both models, but the WGAN outcomes were not very good. Therefore, I tried WGAN-GP, and as expected, it gave better results. The Vanilla GAN produced good output but not the best, as it is considered simple; however, it still provided the expected output according to its capabilities.

Vanilla GAN:

Vanilla GAN is the original Generative Adversarial Network model The Generator creates synthetic data to mimic real data, while the Discriminator tries to distinguish between real and fake samples. This adversarial training helps the Generator improve over time to produce realistic synthetic data.

The architecture:

Vanilla GAN consists of two neural networks: the Generator and the Discriminator. The Generator takes random noise as input and tries to produce synthetic samples that resemble the minority class data. The Discriminator's job is to distinguish between real samples from the dataset and fake samples produced by the Generator. The Generator uses several dense layers with ReLU activations to create synthetic data samples, while the Discriminator uses fully connected layers with LeakyReLU activations to predict whether the input is real or fake.

Training:

```
Epoch 2/1000 [D loss: 0.7207, acc.: 25.00%] [G loss: 0.6881]

Epoch 1000/1000 [D loss: 0.7612, acc.: 6.42%] [G loss: 0.6150]
```

- The drop in Discriminator accuracy from 25% to 6% over training reflects the Generator's success in creating realistic synthetic samples, making the Discriminator less confident in telling fake from real.
- The increase in Discriminator loss and decrease in Generator loss are typical signs that the Generator is "winning" the adversarial game, improving the quality of generated data.
- These trends indicate the Vanilla GAN is training effectively, with the Generator learning to mimic the minority class distribution over time.

WGAN-GP (Wasserstein GAN with Gradient Penalty)

WGAN-GP is an improved GAN variant designed to address training instability and mode collapse issues that often affect Vanilla GANs and even the normal WGAN and CGAN.

It leverages the Wasserstein distance as a measure of divergence between real and generated data distributions

Architecture:

The WGAN-GP consists of two neural networks: the Generator and the Critic. The Generator takes a random noise vector of dimension *X* and produces synthetic samples with the same number of features as the minority class. It contains fully connected layers with increasing complexity starting from 256 units, then 512 units, each followed by LeakyReLU activations and Batch Normalization. The Critic takes samples, either real from the minority class or fake from the Generator, and produces a single linear output neuron providing a real-valued score. It consists of fully connected layers decreasing in size from 512 to 256 units, each followed by LeakyReLU activations.

Training:

```
Epoch 1/1000, Critic Loss: 3.7893, Generator Loss: 0.0446
Generated and clipped samples:
[[  3.913764   151.75143     33.806866    35.438286   119.78513
   44.30458      0.24068497  41.98244   ]
 [  4.9708624  124.942406    40.644566    35.81922     81.60614
   46.03733      1.1818632   23.105707  ]
 [  2.299099   131.64122     50.065796    24.645912   145.6389
   43.83993      1.0551405   38.982887  ]
 [  4.7573233  132.2709      45.06879     39.274532   155.38695
   41.055027     1.4470706   47.183266  ]
 [ 10.152785   120.44343     57.467724    16.974451   187.81075
   42.761497     0.9238229   47.856667  ]]

Epoch 1000/1000, Critic Loss: -0.2977, Generator Loss: -0.2072
Generated and clipped samples:
[[  9.601788   129.06908     72.29743     18.57154    189.34421
   33.67355      0.6799872   34.995697  ]
 [  1.6209308  145.93549     91.24064     23.879608    27.360159
   33.112183     0.520783    33.06242   ]
 [  1.0621767  143.48201     74.10616     42.743286   125.64298
   35.606983     0.27378613  25.945972  ]
 [ 11.244201   155.3709      84.29257     12.779666    32.00303
   30.490353     0.56575805  59.84893   ]
 [  4.943327   144.4226      39.07153      8.772298    62.28851
   28.698832     0.48446977  34.068104  ]]
```

At the beginning (Epoch 1), the Critic loss is high (3.7893), reflecting poor initial discrimination between real and fake samples, while the Generator loss is low (0.0446) as it starts learning to fool the Critic

At (Epoch 1000) the Critic loss stabilizes around negative values (~ -0.3), and the Generator loss varies, indicating the adversarial balance improving. showing increasingly realistic synthetic minority class data when scaled back to original feature ranges.

```
              precision    recall  f1-score   support

           0       0.92      0.86      0.89       100
           1       0.87      0.93      0.90       100

    accuracy                           0.90       200
   macro avg       0.90      0.90      0.89       200
weighted avg       0.90      0.90      0.89       200

ROC-AUC Score: 0.9504
```

LSGAN:

LSGAN improves upon the original GAN framework by using a least squares loss function for the discriminator and generator instead of the traditional binary cross-entropy loss. This approach helps stabilize training and generates higher-quality synthetic samples

Architecture:

LSGAN consists of two neural networks: Generator and Discriminator.the Generator takes Random noise vector of dimension X and produce Synthetic samples matching the minority class features , produced through a tanh activation to scale outputs between -1 and 1.it consists fully connected layers with increasing complexity, starting with 256 units and then 512 units. Each layer uses LeakyReLU activations and Batch Normalization,while the discriminator takes Either real minority class samples or synthetic samples from the Generator to produce A single linear neuron producing a real-valued output used in the least squares loss calculation.it consist of Fully connected layers with decreasing size from 512 units to 256 units, each followed by LeakyReLU activations.

Training Progress:

```
Epoch 1/1000, D Loss: 0.4039, G Loss: 0.3454
Generated samples:
[[  8.813664  108.86315    67.80987    39.267155  542.3554     30.111519
    1.0862076  37.055252 ]
 [  5.7818747 160.68027    51.438766   31.757154  536.9439     47.147354
    1.7523924  27.942268 ]
 [  2.546302  102.284134   43.330856   34.642902  448.84573    38.864254
    1.7283905  36.438736 ]
 [  5.979085  133.40819    79.85057    39.029358  489.99454    32.47015
    1.5212002  35.544746 ]
 [  5.831278  137.73062    28.447968   40.539745  362.38504    32.504387
    1.9540796  47.89629  ]]
Epoch 1000/1000, D Loss: 0.0591, G Loss: 0.3980
Generated samples:
[[2.5508232e+00 1.3963611e+02 5.4039978e+01 2.2942816e+01 7.6455742e+01
  3.5844746e+01 2.9848376e-01 2.4375717e+01]
 [4.8021972e-03 1.6383313e+02 8.0096703e+01 4.7657101e+01 4.6920111e+02
  5.4051266e+01 1.7876397e-01 2.1887409e+01]
 [6.8504748e+00 1.3760272e+02 8.0104218e+01 3.6261536e+01 2.4633543e+02
  3.5743301e+01 7.7878052e-01 4.6862816e+01]
 [1.2495450e+01 1.5212955e+02 8.8978889e+01 4.1442387e+01 1.6210327e+02
  3.0808231e+01 2.4382286e-01 4.2956245e+01]
 [1.6109142e+00 1.1248774e+02 7.0642189e+01 3.1095295e+01 3.0250401e+01
  2.9057505e+01 6.0845602e-01 2.2105030e+01]]
```

At the start (Epoch 1), the Discriminator loss (~0.40) and Generator loss (~0.35) indicate initial learning stages, with the Generator producing rough synthetic samples.

As training proceeds towards 1000 (epochs) the generated samples improve visually and statistically during training. When the synthetic data is rescaled back to the original feature ranges, it more closely resembles the distribution of the minority class. This progressive refinement indicates that the Generator is effectively capturing the underlying data patterns, making the synthetic samples suitable for augmenting the imbalanced dataset.

Classifier Setup and Evaluation

For evaluating the impact of synthetic data generated by different GAN variants, we used a Multi-Layer Perceptron (MLP) classifier. The dataset was split into training and testing sets with an 80-20 ratio.

This classification pipeline was applied on four datasets:

1. The **original imbalanced dataset**.
2. The dataset balanced by augmenting with **Vanilla GAN** synthetic samples.
3. The dataset balanced by augmenting with **LS-GAN** synthetic samples.

4. The dataset balanced by augmenting with **WGAN-GP** synthetic samples.

Evaluation Metrics:

I have used all the evaluation metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix

Results & Comparisons

Original dataset: Accuracy: 0.8417, Precision: 1.0000, Recall: 0.0500, F1-Score: 0.0952, ROC-AUC: 0.5515

Confusion Matrix: [[100 0] [19 1]]

Vanilla GAN Balanced Dataset: Accuracy: 0.7500, Precision: 0.7778, Recall: 0.7000, F1-Score: 0.7368, ROC-AUC: 0.8216

Confusion Matrix:

[[80 20] [30 70]]

LS-GAN Balanced Dataset

Accuracy: 0.7550, Precision: 0.7576, Recall: 0.7500, F1-Score: 0.7538, ROC-AUC: 0.8295

Confusion Matrix:

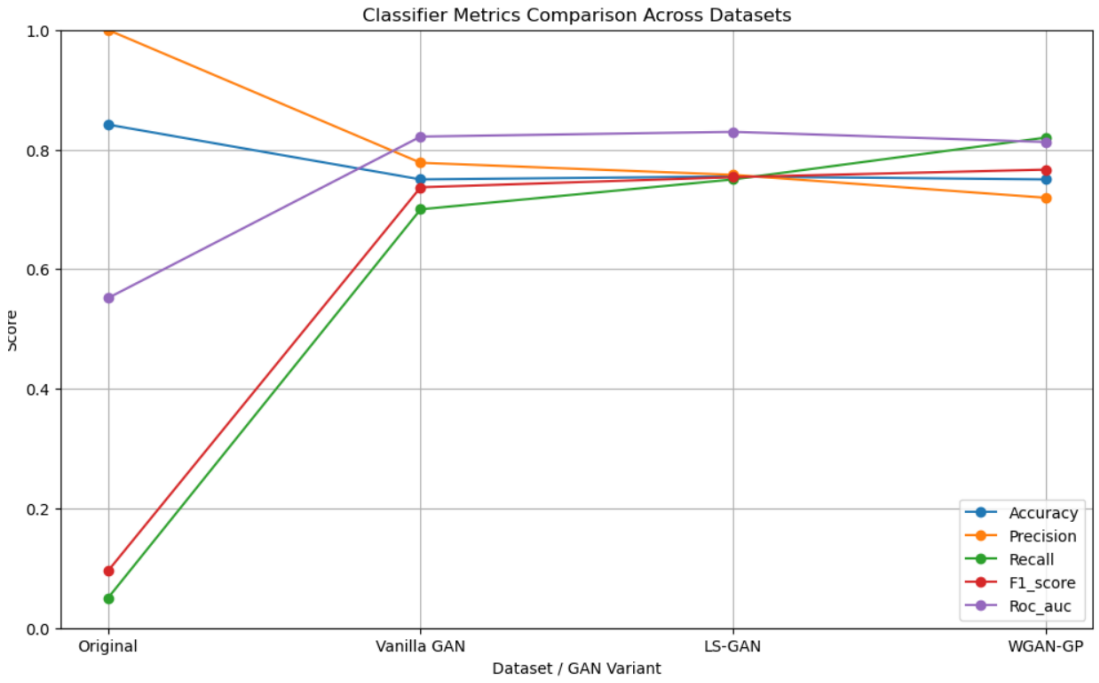
[[76 24] [25 75]]

WGAN-GP Balanced Dataset

Accuracy: 0.7500, Precision: 0.7193, Recall: 0.8200, F1-Score: 0.7664, ROC-AUC: 0.8125

Confusion Matrix:

[[68 32] [18 82]]



The classifier on the original imbalanced data showed relatively high accuracy (84%) but very poor recall (5%) for the minority (patient) class,. Balancing with Vanilla GAN greatly improved recall to 70% and boosted ROC-AUC to 0.82, showing better minority detection. LS-GAN achieved slightly better overall balance with 75.5% accuracy and ROC-AUC of 0.83, while WGAN-GP excelled in recall (82%) crucial for medical diagnosis, with an accuracy of 75% and ROC-AUC near 0.81. Overall, GAN-based augmentation significantly enhanced minority class detection, with LS-GAN and WGAN-GP performing best. Although 75% accuracy with balanced precision and recall is encouraging

Sample of the generated data:

Sample Synthetic Data from Vanilla GAN:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	11.630731	84.373566	72.843674	37.087036	5.719704	31.876221	1.044335	39.677307	1
1	0.111984	144.429672	76.958527	38.526024	131.183716	45.744572	0.476885	24.034443	1
2	0.322717	109.158531	62.107048	36.470547	341.145538	34.008881	0.656734	21.915953	1
3	0.335523	90.937698	81.455765	37.010212	47.128021	43.384190	0.614989	24.582718	1
4	0.032989	168.871201	78.895218	33.535099	346.399384	38.577049	0.326031	26.854830	1

Sample Synthetic Data from LS-GAN:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	4.193018	92.202217	79.417496	35.075680	217.387146	35.240295	0.544526	31.673717	1
1	12.942079	116.796921	82.323326	37.572441	26.891417	44.648811	0.207682	56.385784	1
2	3.555254	105.293694	75.444878	23.174232	54.186432	29.686584	0.514600	45.415958	1
3	3.895673	90.511581	78.528770	34.712589	14.168840	31.900482	1.074989	34.210800	1
4	5.887457	178.742966	84.061028	35.809464	274.747620	32.932362	0.903008	57.816750	1

Sample Synthetic Data from WGAN-GP:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	0.926360	105.586075	69.242783	24.107128	55.930923	31.659515	0.568505	47.638962	1
1	9.879734	96.733368	77.404633	33.717663	3.588456	36.896652	0.283069	46.776344	1
2	1.807072	125.129440	67.287231	27.274605	16.612942	33.619900	0.815892	24.532660	1
3	4.004433	100.771675	73.042526	19.234900	43.681953	29.958916	0.468089	28.956450	1
4	10.383139	124.087181	71.736343	28.400341	111.514267	26.317467	0.317513	44.986652	1

Observations and Conclusions:

In this project, we saw how the original imbalanced dataset made it really hard for the model to correctly identify patients, with very low recall despite decent overall accuracy. Using GANs to generate synthetic data helped balance the classes and significantly improved the model's ability to detect the minority class. Among the GANs, LS-GAN gave the best overall balanced results, while WGAN-GP was especially good at catching more positive cases, which is crucial in medical diagnosis. Vanilla GAN helped too but wasn't as strong as the others. Overall, using GANs proved to be a practical way to tackle imbalance, though the best choice depends on whether you want more balanced predictions or to focus on minimizing missed cases.